

The `_call_` method is called, if the instance is called “like a function”, i.e. using brackets as shown here: <https://dgit.in/cmPy>.

Generators

Generator is also a type of function that uses the `yield` keyword in the body of its code. So what is the `yield` keyword? Like the `return` statements, `yield` also sends a value back to the caller. The difference is, it does not exit the function but can continue where it left off. When resumed, the function continues execution immediately after the last `yield` run. This allows its code to produce a series of values over time, rather than computing them at once and sending them back like a list. Example as shown here: <https://dgit.in/yiePy>.

`Yield` is used when you want to iterate over a sequence, but don't want to store the entire sequence in memory.

The next concept you need to know is iterators. Iterator is any python type that can be used with a ‘for in loop’. Lists, tuples, dictionaries and sets are inbuilt iterators. An iterator uses 2 methods- `__iter__` and `next ( __next__ )`. The first initializes the iterator and is followed by `next` which returns the next iterable value. When an iterator is used with a ‘for in’ loop, the for loop implicitly calls `next ( )` on the iterator object. A `StopIteration` is raised signaling the end of the code.

Let's create a code that creates an iteration from value 20 to a given limit, say 25. Once it reaches 25, it will stop; if the starting number is less than 20, it will not start. Example code as shown here: <https://dgit.in/iteraPy>.

The output will be a series of numbers from 20 to 25 and a `StopIteration` for the second call.

Now combine both `yield` and iterators to create a generator function as shown here: <https://dgit.in/iteraPy2>.

This will produce a reverse string of each character of the word ‘hello’ using the `range` function. Similarly, the generator function can also be used with list, tuple etc. The major difference between a list comprehension and a generator expression is that while list comprehension produces the entire list, generator expression produces one item at a time.

Generators are easy to implement and make for a cleaner, neater code. Take a look at the code shown here: <https://dgit.in/genPy>.

These are just 2 advanced options shown here out of a wide variety. The point is that Python has several such cool features available and one can experiment and play around to understand them all in detail. **d**